From Shiny App to Enterprise SaaS Solution: Lessons Learned and Necessary Tech Stack

Authors: Mauro Masiero, Nicholas Cullen, Xavier Escribà Montagut, Gabriela Scorici, Anastasija Nufer, Carson Sievert[1], Cédric Scherer[2], John Coene[3], Layal Khayal, Marco Sciaini[4], Matt Leech[5], Santiago Muñiz, Shalini Pandurangan, Stefan Reifenberg[4], Axel Martinelli, Murat Akhmedov, Ivo Kwee

Affiliations: 1. Posit/Sievert consulting 2. Cédric Scherer Consulting 3. Opifex 4. Kaldera GbR 5. Matt Leech Consulting

root@bigomics:/#

--R-Shiny enables fast prototyping of simple user interfaces that allows users to explore and bring bioinformatics tools to a broad range of users

--Problem: Shiny apps are often monolithic and run locally (single App.R), along with simple to no deployment structure. This model limits scalability and deployment of high-end software-as-a-service (SaaS) applications.

--framework Here we present the technologies that brought our Shiny App with 80.000+ lines of code to an enterprise-grade solution.

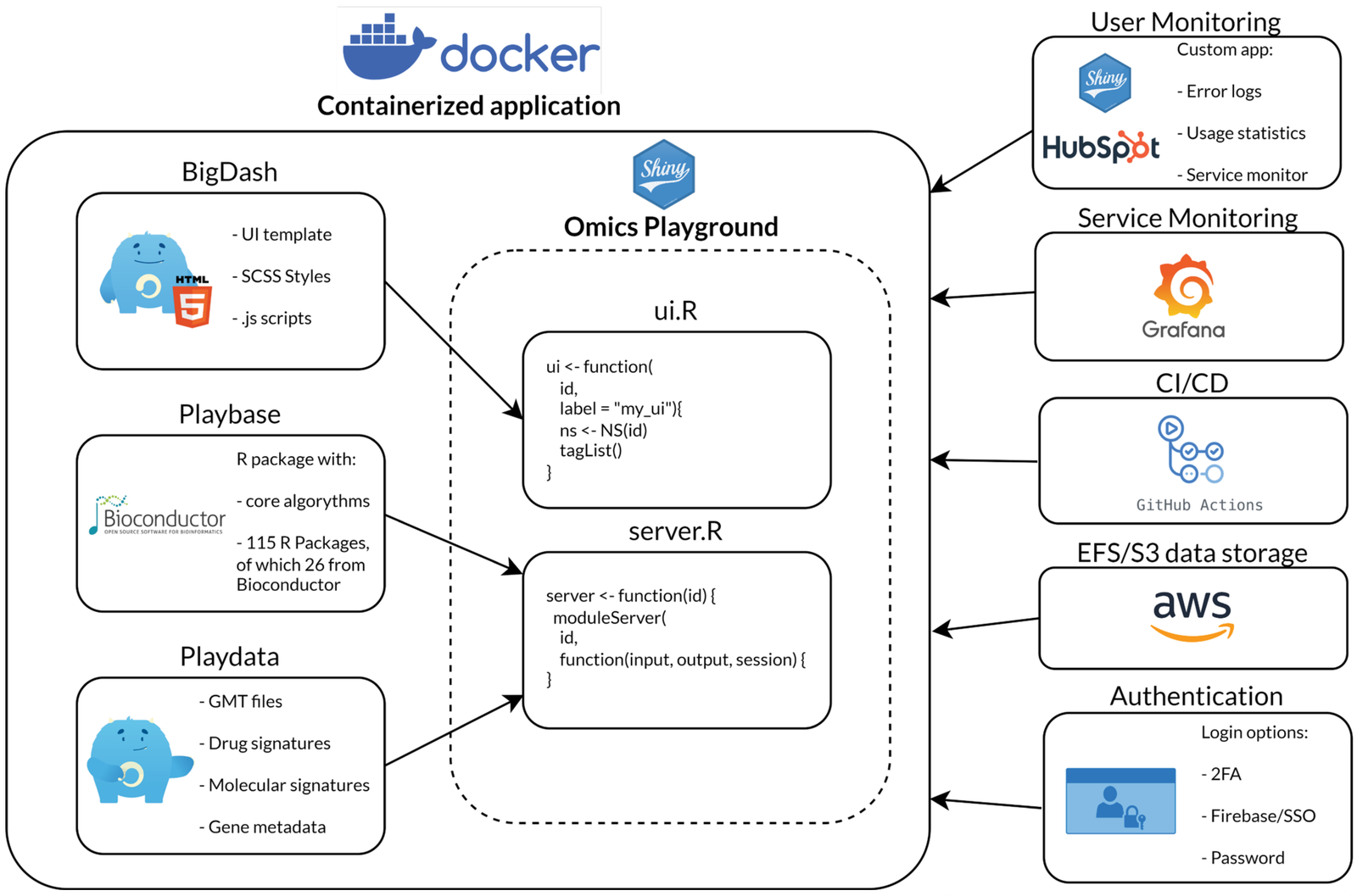

Figure 1. Technology framework used in Omics Playground.

tree omicsplayground/ -d -L 2

To accomodate 18 dashboards and 80.000+ lines of code, we had to organize the folder structure for better readability, management and collaboration. This way each module can be maintained separately.

.bin/ has R or bash scripts to facilitate command line applications.

.components/ has the main ui and server, together with the modules.

.data*/ user, public and shared data.

.dev/ has handy tools to aid development.

.docker/ has the docker files to create the necessary images.

.etc/ contains user/app settings and credentials.

.tests/ contains UI tests, similar to R package unit testing.

```
omicsplayground/
├── bin
├── components
│   ├── app
│   ├── board.1
│   ├── board.n...
│   ├── modules
│   └── ui
├── data
├── data_public
├── data_shared
├── dev
│   └── board.launch
├── docker
├── etc
└── tests
```

Figure 2. Directory tree of Omics Playground.

framework --help

User and service monitoring

A shiny app used to check the status of the deployment, number of users, errors, and other aggregated user data.

CI/CD

From our github codebase, we automate actions to test the code, build the docker images, and deploy them on the servers.

EFS/S3

We use two different storage systems: EFS for user-based files and S3 buckets for data sharing across users and settings.

Authentication

Omics Playground offers SSO or email-based authentication.

docker stack deploy

Deployment utilizes Docker Swarm with Traefik as a reverse proxy to manage and load balance multiple services.

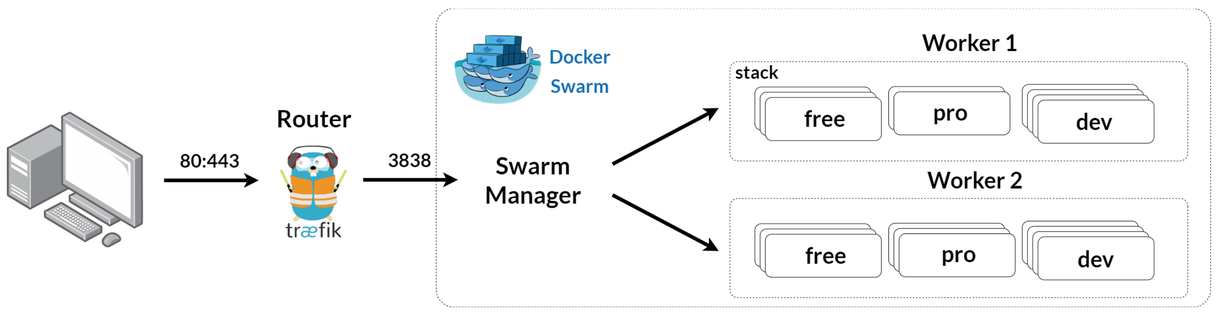

Figure 3. Deployment framework of Omics Playground.

exit 0

--summary Here we introduced the framework for an Enterprise-grade Shiny App. The modular structure enabled complex functionalities such as data sharing, authentication, deployment and seamless integration with over 100 R Packages.

--status Our App is constantly evolving based on user feedback, but also on our own beliefs of how bioinformatics will develop in the future. New technologies are incorporated as they develop.

--next-steps We look forward to discussing solutions for shiny apps, but also integration with Bioconductor packages.

Contact Us

www.bigomics.ch
hello@bigomics.ch
@bigomics

READY TO PLAY?

Try Omics Playground at *bigomics.ch*
or find us on GitHub at *github.com/bigomics*